

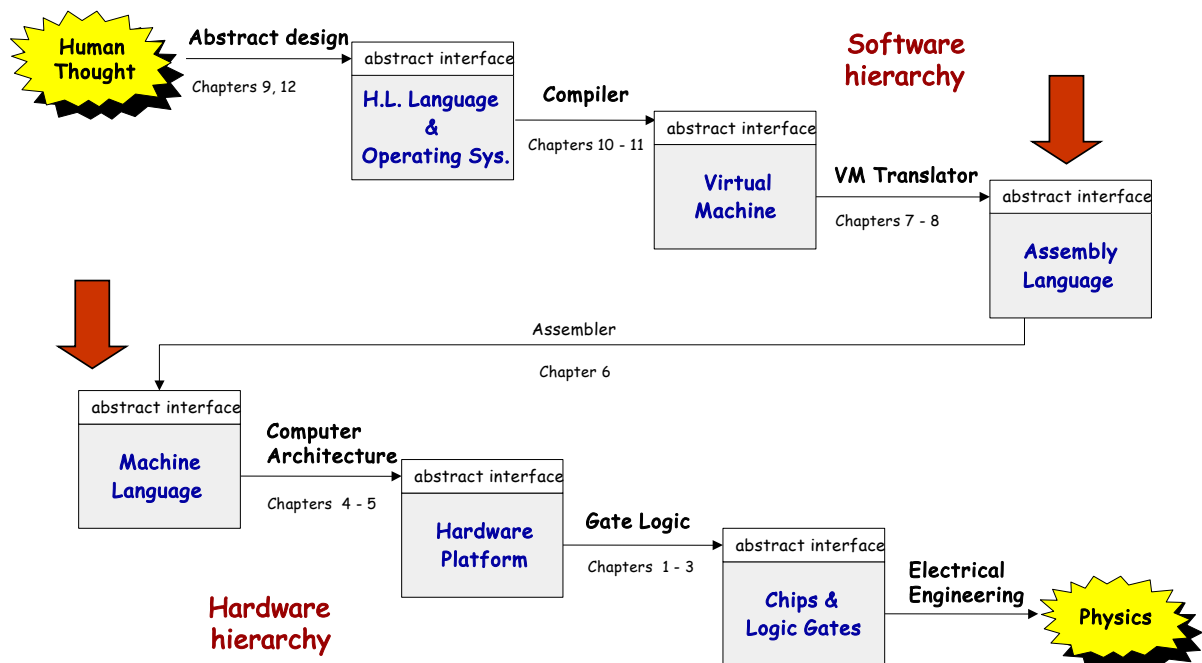
Machine Language



Building a Modern Computer From First Principles

1

Where we are at:



2

Machine language

Abstraction - implementation duality:

- Machine language (= instruction set) can be viewed as a programmer-oriented abstraction of the hardware platform
- The hardware platform can be viewed as a physical means for realizing the machine language abstraction

Another duality:

- Binary version
- Symbolic version

Loose definition:

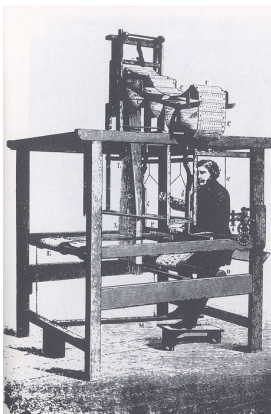
- Machine language = an agreed-upon formalism for manipulating a memory using a processor and a set of registers
- Same spirit but different syntax across different hardware platforms.

3

Binary and symbolic notation

1010 0001 0010 1011

ADD R1, R2, R3



Jacquard loom
(1801)

Evolution:

- Physical coding
- Symbolic documentation
- Symbolic coding
- Translation and execution
- Requires a translator.



Augusta Ada King,
Countess of Lovelace
(1815-1852)

4

Typical machine language commands (a small sample)

```
// In what follows R1,R2,R3 are registers, PC is program counter,  
// and addr is some value.  
  
ADD R1,R2,R3      // R1 ← R2 + R3  
  
ADDI R1,R2,addr   // R1 ← R2 + addr  
  
AND R1,R1,R2      // R1 ← R1 and R2 (bit-wise)  
  
JMP addr          // PC ← addr  
  
JEQ R1,R2,addr    // IF R1 == R2 THEN PC ← addr ELSE PC++  
  
LOAD R1, addr     // R1 ← RAM[addr]  
  
STORE R1, addr    // RAM[addr] ← R1  
  
NOP               // Do nothing  
  
// Etc. - some 50-300 command variants
```

6

The Hack computer

A 16-bit machine consisting of the following elements:

Data memory: RAM - an addressable sequence of registers

Instruction memory: ROM - an addressable sequence of registers

Registers: D, A, M, where M stands for RAM[A]

Processing: ALU, capable of computing various functions

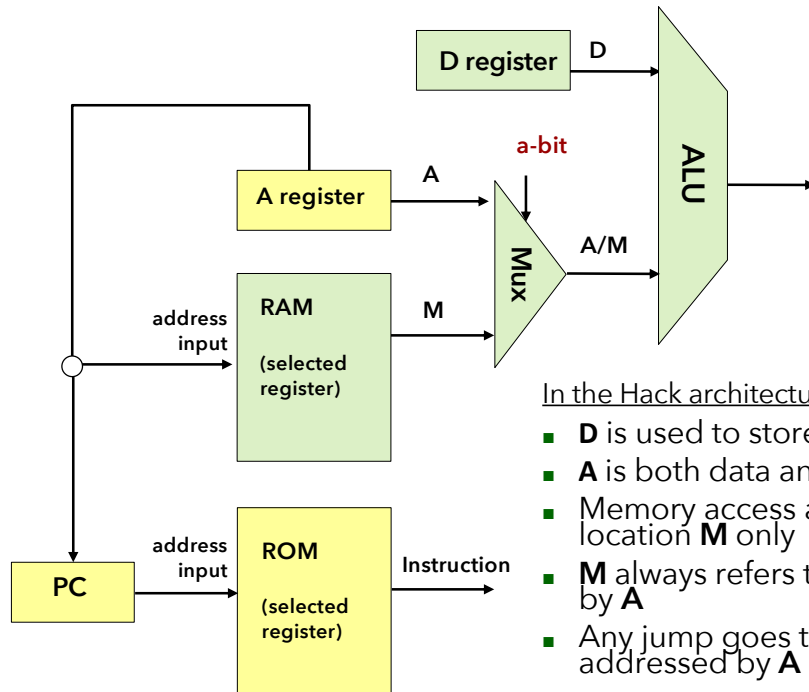
Program counter: PC, holding an address

Control: The ROM is loaded with a sequence of 16-bit instructions, one per memory location, beginning at address 0. Fetch-execute cycle: later

Instruction set: Two instructions: A-instruction, C-instruction.

7

The Hack computer organization



In the Hack architecture:

- **D** is used to store data only
- **A** is both data and address register
- Memory access applies on an implicit location **M** only
- **M** always refers to the location addressed by **A**
- Any jump goes to the instruction addressed by **A**

8

The Hack computer organization

- **D** (data only), **A** (data and address) are manipulated by arithmetic/logical instructions
 - Examples: $A = D - 1$, $D = !A$, ect.
- Memory access applies on an implicit location **M** only
 - Example: $D = M + 1$
- **M** always refers to the location addressed by **A**
 - Example: compute $D = \text{Memory}[516] + 1$
First set $A = 516$, then specify $D = M + 1$
- Any jump goes to the instruction addressed by **A**
 - Example: compute $\text{jmp } 35$ (goto 35)
First set $A = 35$, then **jump**

9

Example - Add integers 1 to 100

C language

```
// Adds 1+...+100.  
int i = 1;  
int sum = 0;  
While (i <= 100){  
    sum += i;  
    i++;  
}
```

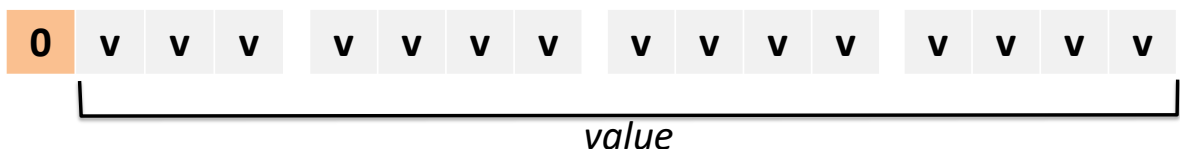
Hack machine language

```
// Adds 1+...+100.  
@i      // i refers to some mem. location.  
M=1     // i=1  
@sum    // sum refers to some mem. location.  
M=0     // sum=0  
(LOOP)  
@i  
D=M     // D=i  
@100  
D=D-A   // D=i-100  
@END  
D;JGT   // If (i-100)>0 goto END  
@i  
D=M     // D=i  
@sum  
M=D+M   // sum=sum+i  
@i  
M=M+1   // i=i+1  
@LOOP  
0;JMP   // Goto LOOP  
(END)  
@END  
0;JMP   // Infinite loop
```

10

A-instructions

- **Syntax:** @*value*
- **Semantic:** set the **A** register to *value*
 - set a constant into the register
 - set the stage for a subsequent C-instruction manipulating data from a memory location
 - set the stage for a subsequent C-instruction specifying a jump to a specific address
- **Binary coding:**

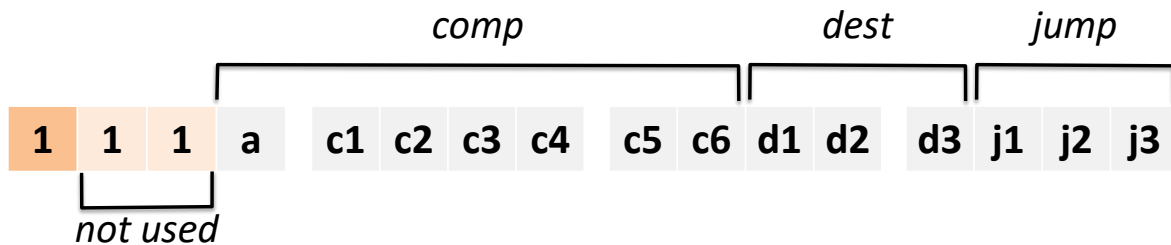


Example: @5 is equivalent to 00000000000000101

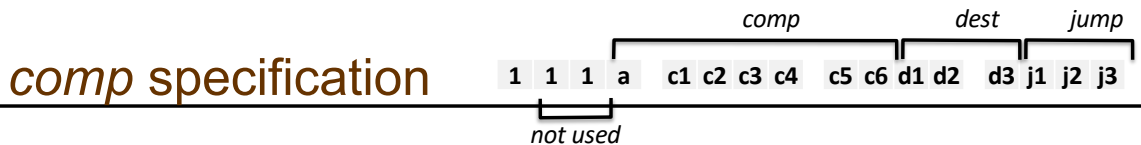
11

C-instructions

- **Syntax:** [*dest=*]*comp*[:*jump*]
- **Semantic:** Answer the questions:
 - what to compute?
 - where to store the computed value?
 - where to go next?
- **Binary coding:**



12



- **comp** = {0, 1, -1, D, A, !D, !A, -D, -A, D+1, A+1, D-1, A-1, D+A, D-A, A-D, D&A, D|A, M, !M, -M, M+1, M-1, D+M, D-M, M-D, D&M, D|M}

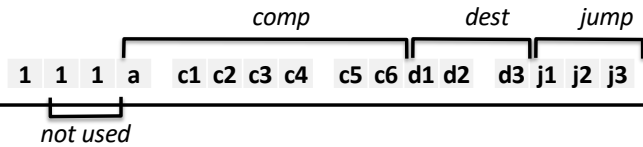
- **Example:** D-1

■ {ac1c2c3c4c5c6}=
0 0 0 1 1 1 0

(when a=0) comp mnemonic	c1	c2	c3	c4	c5	c6	(when a=1) comp mnemonic
0	1	0	1	0	1	0	
1	1	1	1	1	1	1	
-1	1	1	1	0	1	0	
D	0	0	1	1	0	0	
A	1	1	0	0	0	0	M
!D	0	0	1	1	0	1	
!A	1	1	0	0	0	1	!M
-D	0	0	1	1	1	1	
-A	1	1	0	0	1	1	-M
D+1	0	1	1	1	1	1	
A+1	1	1	0	1	1	1	M+1
D-1	0	0	1	1	1	0	
A-1	1	1	0	0	1	0	M-1
D+A	0	0	0	0	1	0	D+M
D-A	0	1	0	0	1	1	D-M
A-D	0	0	0	1	1	1	M-D
D&A	0	0	0	0	0	0	D&M
D A	0	1	0	1	0	1	D M

13

dest specification

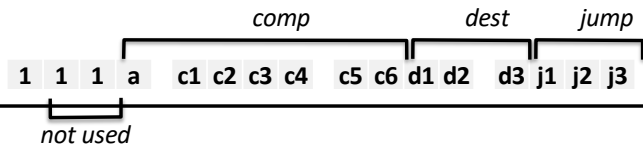


- **dest** = M , D , MD , A , AM , AD , AMD, or null
- Each of the 3 d-bits into the c-instruction (d1, d2, d3) is related to a possible **destination for the computed value** (A, D, M)

d1	d2	d3	Mnemonic	Destination (where to store the computed value)
0	0	0	null	The value is not stored anywhere
0	0	1	M	Memory[A] (memory register addressed by A)
0	1	0	D	D register
0	1	1	MD	Memory[A] and D register
1	0	0	A	A register
1	0	1	AM	A register and Memory[A]
1	1	0	AD	A register and D register
1	1	1	AMD	A register, Memory[A], and D register

14

jump specification



- **jump** = JGT, JEQ, JGE, JLT, JNE, JLE, JMP, or null
- Each of the 3 j-bits into the c-instruction (j1, j2, j3) is related to a **comparison of the computed value with 0** (zero)
 - jump occurs if condition is true

j1 (out < 0)	j2 (out = 0)	j3 (out > 0)	Mnemonic	Effect
0	0	0	null	No jump
0	0	1	JGT	If out > 0 jump
0	1	0	JEQ	If out = 0 jump
0	1	1	JGE	If out ≥ 0 jump
1	0	0	JLT	If out < 0 jump
1	0	1	JNE	If out ≠ 0 jump
1	1	0	JLE	If out ≤ 0 jump
1	1	1	JMP	Jump

15

Symbols and conventions

- Some predefined symbols can be used for the sake of simplicity
 - **Register pointers:** **R0**, **R1**, ..., **R15** refer to RAM addresses 0, 1, ..., 15
 - **I/O pointers:** **SCREEN** and **KBD** refer to RAM addresses 16384 and 24576 (base to screen and keyboard)
 - **Labels:** labels for goto commands can be declared as **(XXX)**
 - **Variables:** symbols for RAM locations (starting from 16) can be declared as **xxx**
- Hack assembly is **case-sensitive**
 - commands and labels: uppercase, variables: lowercase
- Code source text files extension: **.asm**
- Machine language text files (0,1 only) extension: **.hack**